

Translated

From: e-maxx.ru

Placing Bishops on a Chessboard

Find the number of ways to place K bishops on an $N \times N$ chessboard so that no two bishops attack each other.

Algorithm

This problem can be solved using dynamic programming.

Let's enumerate the diagonals of the chessboard as follows: black diagonals have odd indices, white diagonals have even indices, and the diagonals are numbered in non-decreasing order of the number of squares in them. Here is an example for a 5×5 chessboard.

```

$$\begin{matrix} 1 & 2 & 5 & 6 & 9 \\ & 2 & 5 & 6 & 9 & 8 \\ & & 5 & 6 & 9 & 8 & 7 \\ & & & 6 & 9 & 8 & 7 & 4 \\ & & & & 9 & 8 & 7 & 4 & 3 \end{matrix}$$

```

Let $D[i][j]$ denote the number of ways to place j bishops on diagonals with indices up to i which have the same color as diagonal i . Then $i = 1 \dots 2N-1$ and $j = 0 \dots K$.

We can calculate $D[i][j]$ using only values of $D[i-2]$ (we subtract 2 because we only consider diagonals of the same color as i). There are two ways to get $D[i][j]$. Either we place all j bishops on previous diagonals: then there are $D[i-2][j]$ ways to achieve this. Or we place one bishop on diagonal i and $j-1$ bishops on previous diagonals. The number of ways to do this equals the number of squares in diagonal i minus $j-1$, because each of $j-1$ bishops placed on previous diagonals will block one square on the current diagonal. The number of squares in diagonal i can be calculated as follows:

```
int squares (int i) {
    if (i & 1)
        return i / 4 * 2 + 1;
    else
        return (i - 1) / 4 * 2 + 2;
}
```

The base case is simple: $D[i][0] = 1$, $D[1][1] = 1$.